**Project: Semester Project: Smart Parking Platform**

Version: 6.0

Date: 10.15.2026

## 1. Introduction

WE ARE $oftware ¢orp. A fictitious software development studio with unlimited resources and budget. The project involves the planning and management of a software system that helps drivers locate, reserve, and pay for parking spaces for parking garages in real time. The platform will be available through web and mobile applications and will provide parking availability information, navigation assistance, and management tools for parking operators. The goal is to develop a project management plan for the creation of this system, including requirements gathering, stakeholder management, scheduling, budgeting, risk management, and team coordination.

Aiming to do more than our competitors:

Parknav® - Find Real Time On-Street and Off-Street Parking

Just shows parking, cant pay for it ahead of time

SpotHero | Get Everywhere Easier | Reserve Parking and Save
Doesnt show a real time GPS or open parking, you need to know the address before hand

Spot – Your Smart Parking Companion
Has a waitlist to even join the app and not usable on non-mobile devices

Download SpotHero - Find Parking for Android & iOS - Mobexer
Known to only work in small towns with inaccurate pricing info, crashes unexpectedly. And slow support

### 1.1 Purpose

This document defines the requirements for Semester Project: Smart Parking Platform.

### 1.2 Scope

Drivers often spend significant time searching for available parking, leading to congestion, frustration, and lost productivity. Parking operators need better tools to monitor occupancy, manage pricing, and improve utilization of parking assets.

Hopes to do all the competition can do and more

Out of scope would be things that require more time such as:

Communication with other customers through the app, making it a pseudo social media platform.

It's own GPS system requiring dedicated owned satellites without connecting to others

Extra addons without proper time slots

Calling through the app without using the dedicated methods such as the customers' normal phone plan.

## 2. Overall Description

- Users: <List the different groups of users that will be using the finished product>

- System Environment: Web-based / desktop / mobile application

- Constraints: Must run on specific hardware, must require specific authentication, etc.

- Assumptions: Application has internet / intranet access, etc.

## 3. Functional Requirements

The platform should provide:

User registration and authentication

Real-time parking space availability

Interactive map displaying available parking locations

Parking reservation capability

Digital payment processing

Reservation history and receipts

Administrative dashboard for parking operators

Occupancy reporting and analytics

Notifications and alerts for users

Integration with external mapping/navigation services

**4. Non-Functional Requirements**

NFR1: The system shall support at least 1,000 concurrent users.

NFR2: The system shall update parking values within 1 second.

NFR3: The system shall provide 99.99% uptime.

NFR4: The system shall provide a visual parking gps/map

NFR5: The system shall provide support lines for the app or dedicated parking area

NFR6: The system shall be able to save info such as insurance info.

NFR7: The system shall be able to save info such as roadside assistance info.

**5. Use Case Example**

**Case 1:**

Title: Nearest parking search (GPS map)

Actor: Customer

Precondition: User is logged in, using search query in GPS

Steps:

1. User selects filters out the kind of parking they want

2. User searches for said parking

3. User views available parking

4. User chooses the parking

Postcondition: user has selected the parking they desire and can go to it or pay for it

**Case 2:**
Title: Nearest parking search (Catalogue grid)

Actor: Customer

Precondition: User is logged in, using search query in catalogue

Steps:

1. User selects filters out the kind of parking they want

2. User searches for said parking

3. User views available parking

4. User chooses the parking

Postcondition: user has selected the parking they desire and can go to it or pay for it

**Case 3:**
Title: creating an account

Actor: Customer

Precondition: User downloads the app or uses the website

Steps:

1. User clicks "Sign up"

2. User adds their email

3. User verifies their email

4. User creates their username and password

5. User add two factor authentication (optional)

6. User adds payment info during account creation to be saved to said account (optional)

7. User inserts and saved driver information to their account

Postcondition: user has created an account

**Case 4:**

Title: Adding/saves Payment info (internal, before parking)

Actor: Customer

Precondition: User is logged in, has payment info to save

Steps:

1. User selects the setting menu

2. User selects payment options

3. User puts in payment info with secure vendor/ or uses saved data on device

4. User can use said info during parking

Postcondition: user has saved a payment option to their account

**Case 5:**

Title: Adding/saves Payment info (internal, during parking)

Actor: Customer

Precondition: User is logged in, user has the parking they want to pay for in view

Steps:

1. User selects the parking

2. User selects payment options

3. User puts in payment info with secure vendor/ or uses saved data on device

4. User pays for parking

Postcondition: user has saved a payment option to their account

**Case 6:**

Title: Adding/saves insurance for ease of access (optional)

Actor: Customer

Precondition: User is logged in Steps:

1. User selects the account

2. User selects insurance info options

3. User puts in insurance info with a printable file format

4. User can view their insurance by going to their profile (local)

Postcondition: user has saved their insurance files locally for ease of access

**Case 7:**

Title: Viewing parking history

Actor: Customer

Precondition: User is logged in Steps:

1. User selects the account

2. User selects "Parking History"

3. User selects past parking from a list

Postcondition: user will be able to view parking time, date, and payment history

**Case 8:**

Title: Reserving parking

Actor: Customer

Precondition: User is logged in, user has the parking they want to in view

Steps:

1. User selects the parking

2. User selects payment options (optional based on vendor)

3. User chooses to reserve the spot based on the vendor

Postcondition: user has successfully reserved the parking (if allowed by the vendor)

**Case 9:**

Title: Locating route to parking Via GPS

Actor: Customer

Precondition: User is logged in, user has the parking they want in view

Steps:

1. User selects the parking

2. User selects chooses routes based on filters such as tolls and time

3. User can proceed to the parking via the route

Postcondition: user has been routed to the chosen parking

**Case 10:**

Title: Add new vendors

Actor: network admin, Cloud service

Precondition: Vendor is locatable via the web/ database

Steps:

1. User selects available vendors

2. User scans new vendor based on any database they may have

3. User enters the vendor into the application via a dedicated dictionary/table 4. User saves the new vendor with parking being updated for all users

Postcondition: Mor parking/ vendors should show up in the application/ database

**Case 11:**

Title: Remove vendors

Actor: network admin, Cloud service

Precondition: Vendor is locatable via the web/ database

Steps:

1. User selects desired vendors from the database

2. User removes said vendors

Postcondition: the database should update removing the vendor and their parking

**Case 12:**

Title: Display unaffiliated Vendors

Actor: network admin, Cloud service

Precondition: Vendor is locatable via the web/ database

Steps:

1. User selects available vendors database

2. User scans for available non associated parking vendors

3. User chooses to toggle on the ability to use said vendors for the application

Postcondition: More parking/ vendors should show up as unaffiliated in the application/ database (optional, client-side toggle for customers as well)

**Case 13:**

Title: Display unaffiliated Vendors

Actor: Customer

Precondition: user is logged in Steps:

1. User selects the setting menu

2. User scans for available non associated parking vendors

3. User chooses to toggle on the ability to use said vendors for the application
   Postcondition: More parking/ vendors should show up as unaffiliated in the application/ database

**Case 14:**

Title: Save local cache and map

Actor: Customer

Precondition: user is logged in has loaded a local map of available parking before

Steps:

1. User selects the setting menu

2. User scans for available non associated parking vendors

3. User saves a snapshot of local parking and their prices

Postcondition: Parking should be saved locally in case the user loses access to internet. (note, values will not update)

**Case 15:**

Title: Call for roadside assistance

Actor: Customer

Precondition: user is logged, user has saved roadside assistance info to their account

Steps:

1. User selects their profile

2. User selects the insurance tab

3. User selects the roadside assistance tab

4. User can call for roadside assistance/ view their number (mobile only)

5. User can send roadside assistance their location via text (mobile only) Postcondition:

   Roadside assistance will be notified

**Case 16:**

Title: Account deletion

Actor: Customer  Precondition:

User is logged in

Steps:

1. User selects their profile
2. User selects their profile settings
3. User selects delete account
4. User confirms account deletion after re logging the password and verifying through email

Postcondition: The users account will be deleted.

**Part 2 down:**

**1. Frontend**

1.1 Account Management

- Sign Up
- Email verification
- Username/password creation
- 2FA (optional)
- Account Deletion
- Password confirmation
- Email confirmation

1.2 Parking Search

- GPS Map Search
- Parking filters
- Available parking display
- Catalogue Grid Search
- Parking filters
- Grid view display

1.3 Parking Selection

- Parking details
- Parking availability
-

- Vendor information

1.4 Navigation  GPS routing

- Route generation • Toll avoidance
- Travel time filters

1.5 User Profile

- Driver information
- Insurance storage
- Parking history
- Roadside assistance

1.6 Settings

- Payment management
- Unaffiliated vendor toggle
- Offline map cache

**2. Payment System**

2.1 Payment Methods

- Add payment information
- Edit payment information
- Remove payment information

2.2 Parking Transactions

- Pay for parking
- Save payment during checkout
- Payment confirmation

2.3 Reservation Processing

- Vendor reservation support
- Reservation confirmation

-

## 3. Backend Services

### 3.1 Authentication

- Login
- Email verification
- Password management  Two-factor authentication

### 3.2 User Data Management

- Customer profiles
- Driver records
- Insurance documents
- Parking history records

### 3.3 Parking Database

- Parking inventory
- Vendor records
- Parking availability

### 3.4 Map Services

- GPS integration
- Route calculation
- Offline cache synchronization

## 4. Vendor Management

### 4.1 Vendor Administration

- Add vendors
- Update vendor data
- Remove vendors

### 4.2 Vendor Discovery

- Scan for new vendors
- Database synchronization

### 4.3 Unaffiliated Vendors

-

- Detect unaffiliated vendors
- Enable vendor visibility
- Customer/vendor toggle controls

**5. Cloud & External Integrations**

5.1 Payment Gateway  Secure payment processing

-

- Transaction validation

5.2 Mapping Services

- GPS service integration
- Route generation

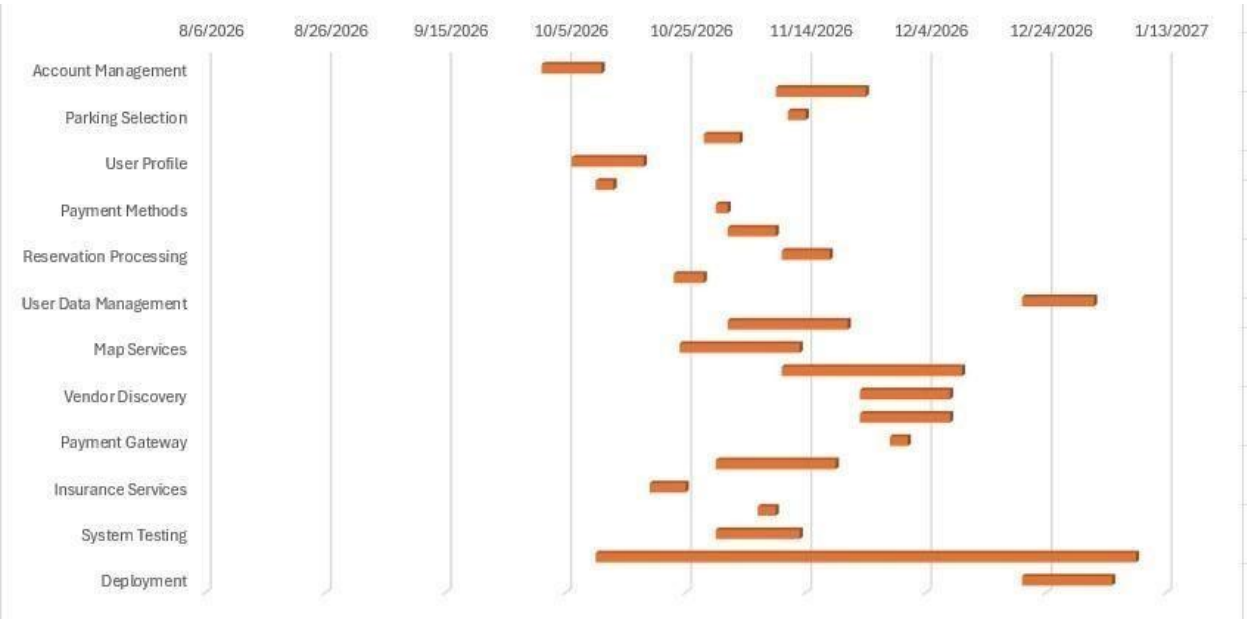
5.3 Insurance Services

- Insurance file storage
- Roadside assistance data

5.4 Notification Services

- Email verification
- Account deletion confirmation
- Reservation confirmations

**Gant Chart timeline (based on WBS):**




**Trello Board:**




**1. Risk Identification**

**Technical Risks**

1. **Update Latency**: Latency when update parking due to high server load or network congestion.
2. **Third Party/ Vendor API Outages**: Disruptions or rate limits on mapping, GPS, or payment gateway APIs.

3. **Database Scalability** : Possibility of backend databases being unable to handle too many concurrent users simultaneously.
4. **Possible Offline Cache Inconsistencies**: Conflicts with what is saved and what the client side machines display (capabilities).

**Schedule Risks**

1. **Scope Creep**: Adding unplanned during sprint cycles. Such as a vendor offering something that is not accounted for.
2. **Integration Delays**: Delays in connecting frontend components with backend services and external APIs. (People/Technical)
3. **Vendor side issues**: Slower than expected discovery and registration of thirdparty parking vendors. (example, obtaining their parking data and locations)
4. **Issues during Testing**: More testing required due to unexpected bugs during System Testing prior to deployment/ launch.

**Financial Risks**

1. **API Usage Cost Overruns**: High costs associated with mapping services, payment processor fees, or cloud storage.
2. **Budget Exceeded Estimated Development (and buffer)**: Custom mobile/web development expenses exceeding the $120,000–$220,000 estimated market range. (Especially with how volatile the tech industry is currently)
3. **Infrastructure Cost Scaling Unexpectedly**: Cloud infrastructure costs required to sustain 99.99% uptime that may exceed what was estimated.
4. **Loss of Revenue from System Downtime**: Revenue loss resulting from downtime during payment processing or reservation workflows. (likely due to third party vendors in most cases) **People Risks**

1. **Key Personnel Turnover**: Loss of core developers or lead architects during critical development phases. (sickness, natural disasters, etc.)
2. **Skill Gap**: Lack of team experience with certain aspects they may come across.
3. **Communication issues**: Poor collaboration between frontend UI/UX teams, backend developers, and vendor.
4. **Team Burnout**: Workload pressure that could be caused by the 4 month timeline.

**Risk Register:**

| ID | Description | Categroy | Probability |
|---|---|---|---|
| R1 | Latency between updates | Technical | Medium |
| R2 | GPS map outages | Technical | Low |
| R3 | System uptime (possibility of going down | Technical | Low |
| R4 | Payment failures | Technical | Medium |
| R5 | Scheduling delays on Map/GPS services (map provider side) | Schedule | Medium |
| R6 | Scheduling Delays on parking vendor services (vendor side) | Schedule | High |
| R7 | Development cost exceeding resonable estemated budget | Financial | Medium |
| R8 | Cloud services cost scaling unexpectedly | Financial | Medium |
| R9 | Developer scheduling conflics and turnovers | People | Low |
| R10 | Communication issues between frontend, backend, and UI teams | PEople | Medium |
| R11 | Natural Disasters | Technical | High |
| R12 | Pulled funding from the project | Fincancial | MEdium |
| R13 | account for unforseen variables | Schedule | Low |

**3. Communication Plan**

**Meeting Cadence per Team, Reporting Methods**

- **Daily Standup (All Technical Teams: Frontend, Backend, Integrations, UI/UX)**
- **Cadence**: Business days, 15 minutes.
- **Focus**: Daily progress, immediate blockers, and daily tasks (e.g., Trello updates).
- **Sprint Planning and Review (Core Project Team & Stakeholders)**
- **Cadence**: every other week, 1 hour.
- **Focus**: Sprint backlog refinement, demonstration of completed modules, and milestone review.
- **Vendor Synchronization (Vendor communications and System Admins)**
- **Cadence**: Weekly, 30 minutes.
- **Focus**: Vendor integration progress, database synchronization, and onboarding issues.
- **Steering Committee / Executive Briefing (Project Manager & Execs)**
- **Cadence**: Monthly, 45 minutes.
- **Focus**: High-level budget tracking, schedule milestones (Gantt chart status), and risk review.

**Reporting Methods**

- **Project Dashboard (Trello)**: see “Tello Board”
- **Weekly Status Report**: Written email(if this was real)/PDF summary delivered every Friday covering milestone burn-down, financial expenditure, and updated risk status. (such as what we are currently doing now)
- **Risk Escalation**: Automated alerts via Slack/Email for critical downtime or severe risks requiring immediate action. (AWS alarms etc.)

| Roles & Resources | | | | | |
|---|---|---|---|---|---|
| **Role** | **Name** | **FTE %** | **Estimated** | **Hourly Rat** | **Cost** |
| **Project Manager** | Alice | 0.5 | 320 | $65 | $20,800 |
| **Lead Backend Architect** | Bob | 1 | 640 | $85 | $54,400 |
| **Frontend UI/UX Developer** | Carol | 1 | 640 | $70 | $44,800 |
| **Cloud Infrastructure Engineer** | Dave | 0.4 | 256 | $80 | $20,480 |
| **API Integration Specialist** | Frank | 0.5 | 320 | $75 | $24,000 |
| **QA / Database Systems Tester** | Dan | 0.5 | 320 | $50 | $16,000 |
| **Total** | | | **2,496** | | **$180,480** |

| Resource & Cost PLan | | | | | | |
|---|---|---|---|---|---|---|
| **Item** | **Category** | **Qty/Mont** | **Unit Cost** | **Duration (** | **Total** | **Notes** |
| **Core Software Engineering** | Labor | 2,496 hou | Varies | 4 | $180,480 | Roles and resources total |
| **AWS Cloud** | Infra | 4 months | $1,200 | 4 | $4,800 | Cloud to support uptime |
| **GPS Geolocation/Mapping Licensing** | Tools | 4 months | $800 | 4 | $3,200 | Realtime tracking and route calculations |
| **Secure Payment Processor Gateways** | Tools | 4 months | $500 | 4 | $2,000 | Digital payment authorization endpoints |
| **Developer Environments/IDE Keys** | Tools | One time purchase | $1,500 | N/A | $1,500 | Initial cost for environments (estimated) |
| **Unexpected Scope/Latency Contingency** | Reserve | 10% | Applied | N/A | $19,198 | Financial safety cushion for sudden delays |
| **Total** | | | | | **$211,178** | total cost estimation |

| RACI Matrix | | | | | |
|---|---|---|---|---|---|
| Item | Month 1 (Sep) | Month 2 (Oct) | Month 3 (Nov) | Month 4 (Dec/Jan) | Total |
| Developer Labor | $45,120 | $45,120 | $45,120 | $45,120 | **$180,480** |
| Cloud Hosting | $1,200 | $1,200 | $1,200 | $1,200 | **$4,800** |
| Licenses and API Gateways | $6,700 | $1,300 | $1,300 | $1,300 | **$10,600** |
| Contingency | $0 | $4,000 | $6,000 | $9,198 | **$19,198** |
| Total | **$53,020** | **$51,620** | **$53,620** | **$52,818** | **$211,178** |

Market rates: Estimated total cost: 120,000$ - 212,000$ (budget buffer included) and risk assessment sources:

Auto GPS tracker app: Cost for bespoke development

How to Create an App and Make Money in 2026 (Complete Guide)

Smart Parking App Development Cost Breakdown

https://www.stakeholdermap.com/risk/register-common-project-risks.php

https://www.betterway.dev/posts/the-structure-roles-and-process-of-a-high-performingdevelopment-team

https://www.coursera.org/articles/what-is-developmentteam?msockid=022b35e40c62670a373d23750d306661
https://instituteprojectmanagement.com/blog/what-is-the-responsibility-assignment-matrix/

https://www.larksuite.com/en_us/blog/raci-matrix